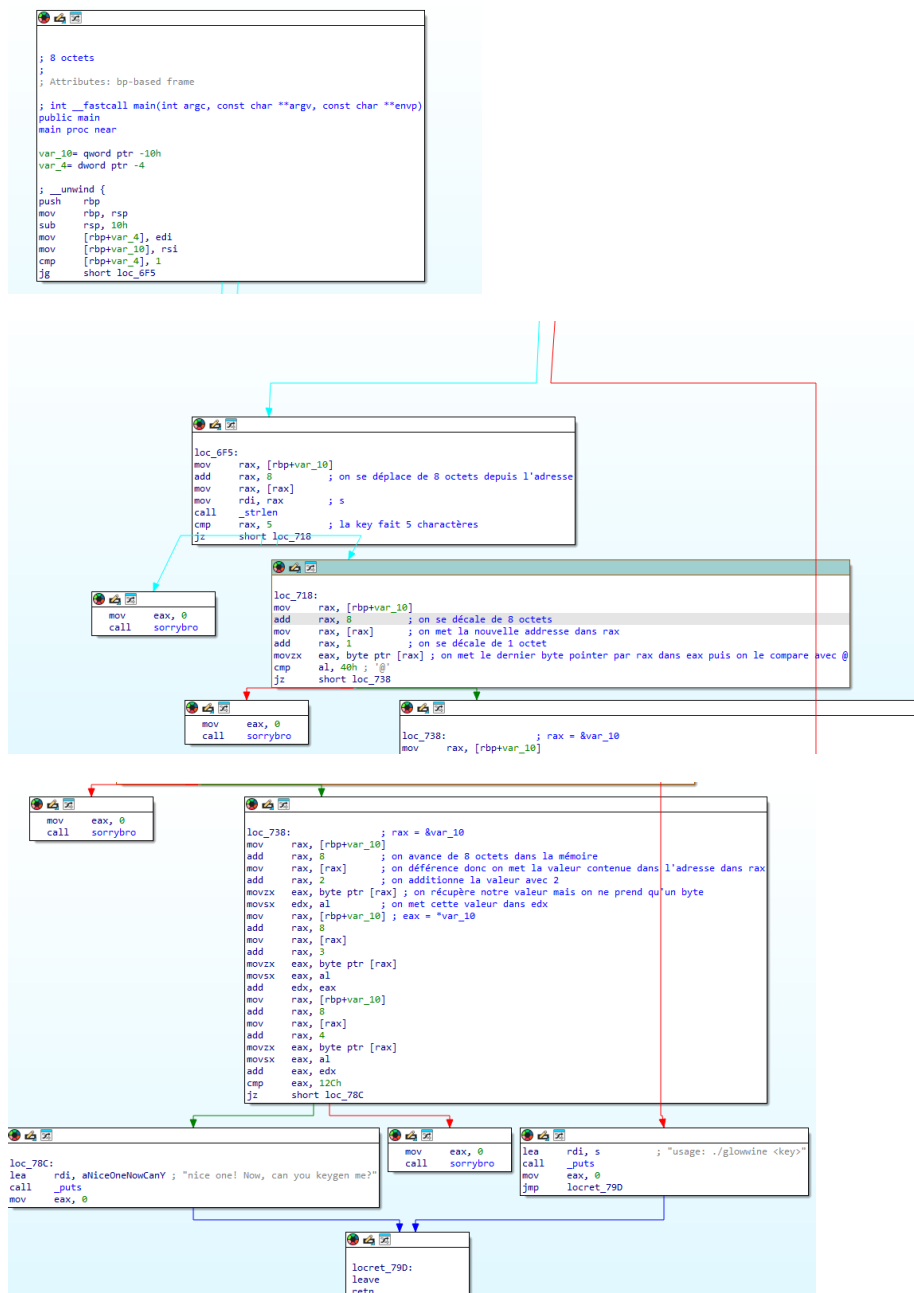


Bkamp's glow wine [keygen practice]

dimanche 31 mai 2026 21:21

<https://crackmes.one/crackme/5df26b4033c5d419aa013362>

```
(agonist@attack)-[~/Bureau/ctf/cracking/glow wine]
$ ./glowwine
usage: ./glowwine <key>
```



Bon on voit que le programme initialise deux variables :
Vars_10 et vars_4.

Var_10 est de type qword donc c'est probablement un pointeur.

Var_4 est de type dword donc c'est probablement un int.

```
var_10= qword ptr -10h
var_4= dword ptr -4
```

Comme var_10 est un pointeur ça veut dire que quand on voit cette instruction `mov rax, [rbp+var_10]` on met dans rax la valeur contenue dans cette adresse.

Ensuite on remarque un autre déréfencement avec `mov rax, [rax]`.

Donc on peut penser que var_10 est un pointeur de pointeur. (merci claude)

A chaque fois on se décale de 8 octets. Donc on va vers une autre adresse d'un autre pointeur.

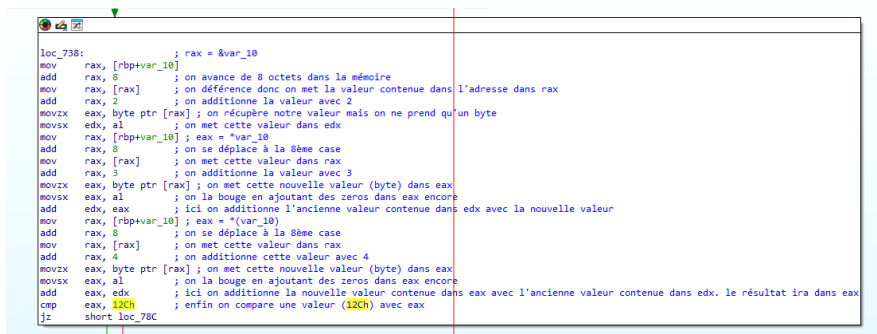
Et quand on voit ensuite le `add rax, 8` ça veut dire qu'on se déplace de 8 octets. Juste après on a un déréfencement `*[rax]`.

Rax contient donc la valeur. Ensuite on fait un `+ 2` et cette nouvelle valeur sera enregistrée dans `edx`.

```

mov     rax, [rbp+var_10]
add     rax, 8           ; or
mov     rax, [rax]       ; or
add     rax, 2           ; or
movzx   eax, byte ptr [rax]
movsx   edx, al          ; or

```



Le programme compare trois arguments qu'on aura rentré avec le résultat 300.

Il pointe d'abord sur notre argv[1] puis avec le add rax, 8 on bouge vers le pointeur numéro 2 (car un pointeur = 8 octets).

Si on additionne en ascii d avec lui-même trois fois on obtient 300.

```

(agonist@attack):[~/Bureau/ctf/cracking/glow wine]
$ ./glowwine ddd
wrong key, try again :/

```

Humm...

En regardant le début j'avais oublié que le programme compare si notre deuxième argument est un @ ou pas.

Ce qui nous donne donc en sachant que notre clé doit faire une longueur de 5 :

```

(agonist@attack):[~/Bureau/ctf/cracking/glow wine]
$ ./glowwine d5ddd
nice one! Now, can you keygen me?

```